УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Математические операции, ввод-вывод, форматирование строк (f-строки), приведение и валидация типов

ЛЕКЦИЯ 3

Ахметов Ильнур Вазирович

ПЛАН

Цели лекции

1. Выполнять вычисления: от простых операторов до модуля math.
2. Принимать данные через input() и выводить через print().
3. Форматировать строки; основной способ — f-строки.
4. Приводить значения к нужному типу: int(), float(), str().
5. Проверять (валидировать) введённые данные.
6. Написать консольный калькулятор скважины с отчётом.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы знаем из лекции 2

- Python — язык с динамической типизацией.
- Переменная — имя, ссылающееся на объект: x = 42.
- Базовые типы: int, float, str, bool, None.
- Тип определяется значением: x = 5 → int, x = 5.0 → float.
- type() показывает тип: type(42) → <class 'int'>.

консоль Python

```
>>> x = 42
>>> type(x)
<class 'int'>
>>> x = 42.0
>>> type(x)
<class 'float'>
>>> name = "Скважина №101"
>>> type(name)
<class 'str'>
```

Арифметические операторы

`+` сложение — 5 + 3 = 8

`-` вычитание — 5 − 3 = 2

`*` умножение — 5 * 3 = 15

`/` деление — 5 / 2 = 2.5 (всегда float!)

`//` целочисленное — 5 // 2 = 2

`%` остаток — 5 % 2 = 1

`**` степень — 5 ** 2 = 25

консоль Python

```
>>> # дебит за 30 дней
>>> total = 1250      # т
>>> days = 30
>>> total / days
41.666666666666664
>>> total // days
41
```

// и % : деление нацело и остаток

- // — деление нацело: сколько целых раз делитель помещается.
- 17 // 5 = 3 17 % 5 = 2
- Проверка: делимое = (частное × делитель) + остаток.
- 17 = (3 × 5) + 2
-
- 1000 т нефти, цистерна 75 т:
- полных цистерн 1000 // 75 = 13
- остаток 1000 % 75 = 25 т

консоль Python

```
>>> total_oil = 1000
>>> tank = 75
>>> total_oil // tank
13      # полных цистерн
>>> total_oil % tank
25      # осталось, т

>>> 17 // 5, 17 % 5
(3, 2)
```

** и pow(): возведение в степень

- a ** b — a в степени b: 2 ** 10 = 1024, 3 ** 4 = 81.
- pow(a, b) — то же: pow(2, 10) = 1024.
- pow(a, b, m) — степень по модулю (криптография).
- Извлечение корня — дробная степень:
- 9 ** 0.5 = 3.0 27 ** (1/3) = 3.0

консоль Python

```
# квадрат и куб давления
>>> P = 15.2   # МПа
>>> P ** 2
231.04
>>> P ** 3
3511.808

>>> P ** 2 ** 0.5
15.2
```

Приоритет операций и скобки

1. ( ) скобки 2. ** степень
3. +x, -x унарные 4. *, /, //, %
5. +, - сложение

2 + 3 * 4 = 14 (не 20!)

(2 + 3) * 4 = 20

Правило: сомневаетесь — ставьте скобки.

порядок вычислений

```
>>> 10 + 5 * 2 ** 3
50      # 2**3=8, 5*8=40, 10+40

>>> ((10 + 5) * 2) ** 3
27000   # 15*2=30, 30**3

>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
```

Сокращённые операторы присваивания

- x += 5 ≡ x = x + 5 x -= 5 ≡ x = x − 5
- x *= 5 ≡ x = x * 5 x /= 5 ≡ x = x / 5
- x //= 5 ≡ x = x // 5 x %= 5 ≡ x = x % 5
- x **= 5 ≡ x = x ** 5
- Зачем: меньше писать, переменная вычисляется один раз.

накопление добычи

```
>>> daily = [12.5, 11.8, 13.1, 12.9]
>>> total = 0
>>> for d in daily:
...     total += d
...
>>> total
50.3
```

Модуль math

- import math — подключаем модуль.
- Константы: math.pi, math.e.
- Корень: math.sqrt(x). ceil() — вверх, floor() — вниз.
- Факториал: math.factorial(5) = 120.
- sin/cos/tan — аргумент в РАДИАНАХ.
- abs(-42) = 42 — встроенная, вне math.

консоль Python

```
>>> import math
>>> math.ceil(12.3)
13
>>> math.floor(12.9)
12
>>> math.sqrt(2)
1.41421356...
>>> math.pi
3.14159265...
```

Округление: round(), int(), ceil(), floor()

- round(x) — математическое округление: round(3.9) = 4.
- int(x) — отброс дробной части: int(3.9) = 3.
- math.ceil(x) — всегда вверх: ceil(3.1) = 4.
- math.floor(x) — всегда вниз: floor(3.9) = 3.
- Не путайте: int() НЕ округляет.

консоль Python

```
>>> round(3.9)
4
>>> int(3.9)
3        # отсечение
>>> import math
>>> math.ceil(3.1)
4        # вверх
>>> math.floor(3.9)
3        # вниз
```

Объём добычи и обводнённость

- Скважина дала 450 т нефти и 150 т воды за месяц.
- 1) общий объём жидкости = oil + water;
- 2) доля нефти = (oil / total) × 100;
- 3) обводнённость = (water / total) × 100.
- Обводнённость (water cut) — ключевой показатель скважины.

консоль Python

```
>>> oil, water = 450, 150
>>> total = oil + water
>>> total
600
>>> oil / total * 100
75.0    # доля нефти, %
>>> water / total * 100
25.0    # обводнённость, %
```

print(): вывод на экран

- print(объект1, объект2, ..., sep=' ', end='\n')
- sep — разделитель между объектами (по умолчанию пробел).
- end — что вывести в конце (по умолчанию перевод строки).
- print("Скважина", "№101") → Скважина №101

консоль Python

```
>>> print("Скважина", "№101")
Скважина №101
>>> print("Ожидание", end="...")
Ожидание...
```

print(): параметры sep и end

- sep — что вставить МЕЖДУ аргументами:
- print(2025, 3, 15, sep="-") → 2025-3-15
- end — что вывести ПОСЛЕ всего:
- print(i, end=", ") — печать в одну строку.
- Пример: строка отчёта по скважине.

консоль Python

```
>>> print(2025, 3, 15, sep="-")
2025-3-15

>>> well, rate = 101, 12.5
>>> print("Скв.", well, "|",
...       "Дебит:", rate, "т/сут")
Скв. 101 | Дебит: 12.5 т/сут
```

input(): ввод данных

- input() ожидает ввод и нажатие Enter.
- переменная = input("Подсказка: ")
- Программа приостанавливается.
- Всё, что ввёл пользователь, возвращается как строка str.
- Даже число 42 приходит как строка "42", а не число 42.

консоль Python

```
>>> name = input("Имя скважины: ")
Имя скважины: СКВ-202
>>> type(name)
<class 'str'>
>>> rate = input("Дебит: ")
Дебит: 12.5
>>> type(rate)
<class 'str'>   # число, но str
```

input() всегда возвращает строку

- Ошибка новичка: арифметика со строкой.
- a = input(); b = input(); print(a + b)
- ввод 10 и 20 → 1020 (конкатенация!), а не 30.
- Правильно — преобразовать строку в число:
- int(input(...)), float(input(...)).

консоль Python

```
# НЕПРАВИЛЬНО
>>> a = input("a: ")
a: 10
>>> b = input("b: ")
b: 20
>>> print(a + b)
1020     # конкатенация!

# ПРАВИЛЬНО
>>> a = int(input("a: "))
>>> b = int(input("b: "))
>>> print(a + b)
30
```

input() + расчёт: калькулятор дебита

float объём нефти может быть дробным.

int количество дней — целое.

расчёт дебит = объём / дни.

вывод форматированный отчёт (f-строка).

код

```
total = float(input("Объём нефти, т: "))
days = int(input("Количество дней: "))

daily_rate = total / days

print("СРЕДНЕСУТОЧНЫЙ ДЕБИТ")
print(f"Объём:    {total} т")
print(f"Дни:      {days} сут")
print(f"Ср.дебит: {daily_rate:.1f} т/сут")
```

Три способа вставить значение в строку

1. Конкатенация: "Дебит: " + str(rate) + " т/сут".
2. Метод .format(): "Дебит: {} т/сут".format(rate).
3. f-строки: f"Дебит: {rate} т/сут".

Изучаем все три, используем f-строки — современный стандарт.

консоль Python

```
rate = 12.5
well = "СКВ-101"

# 1. конкатенация
s1 = "Скв.: " + well

# 2. .format()
s2 = "{}: {} т/сут".format(well, rate)

# 3. f-строка
s3 = f"{well}: {rate} т/сут"
```

Конкатенация и её ограничения

- Конкатенация — склейка строк оператором +.
- "Hello, " + "world!" → "Hello, world!"
- Работает только со строками!
- "Дебит: " + 12.5 → ОШИБКА TypeError
- "Дебит: " + str(12.5) → "Дебит: 12.5"
- Много переменных → нечитаемый код; нет формата числа.

консоль Python

```
# работает
>>> "Скважина" + " №" + "101"
'Скважина №101'

# ОШИБКА
>>> "Дебит: " + 12.5
TypeError: can only concatenate
 str (not "float") to str
```

Метод .format()

- Строка с {} — аргументы подставляются по порядку.
- "{} + {} = {}".format(2, 3, 5) → "2 + 3 = 5"
- По имени: "{name}: {value}".format(name="P", value=15.2).
- По индексу: "{1} - {0}".format(2025, 2026) → "2026 - 2025".
- Формат числа: "{:.2f}".format(42.567) → "42.57".

консоль Python

```
>>> "Дебит: {} т/сут".format(12.5)
'Дебит: 12.5 т/сут'

>>> "Обводнённость: {:.1f}%".format(25.78)
'Обводнённость: 25.8%'
```

f-строки: основной способ

- Строка с префиксом f: всё в {} вычисляется и вставляется.
- f"{well}: дебит {rate} т/сут"
- Преимущества:
- • короче — не нужен .format() в конце;
- • нагляднее — переменная на своём месте;
- • быстрее — компилируются интерпретатором.

консоль Python

```
>>> well = "СКВ-101"
>>> rate = 12.5
>>> f"{well}: дебит {rate} т/сут"
'СКВ-101: дебит 12.5 т/сут'

>>> f"2 + 2 = {2 + 2}"
'2 + 2 = 4'
```

В {} можно любое выражение

- Внутри {} — арифметика, вызов функции, условие.
- f"{oil + water} т"
- f"√2 ≈ {math.sqrt(2):.4f}"
- f"{x} — {"чётное" if x%2==0 else "нечётное"}"
- Не перегружайте скобки — выносите расчёт в переменную.

консоль Python

```
>>> oil, water = 450, 150
>>> f"Объём: {oil + water} т"
'Объём: 600 т'

>>> import math
>>> f"√2 ≈ {math.sqrt(2):.4f}"
'√2 ≈ 1.4142'
```

Форматирование чисел: точность :.Nf

- {значение:.Nf} — N знаков после запятой.
- pi = 3.14159...; f"{pi:.2f}" → "3.14"
- f"{pi:.4f}" → "3.1416"   (округление!)
- :.Nf ОКРУГЛЯЕТ, а не обрезает.
- 42.678 → :.1f → 42.7   :.0f → 43
- Дебит, давление, обводнённость — с разумной точностью.

консоль Python

```
>>> rate = 12.56789
>>> f"Дебит: {rate:.1f} т/сут"
'Дебит: 12.6 т/сут'

>>> pressure = 15.234
>>> f"Давление: {pressure:.2f} МПа"
'Давление: 15.23 МПа'
```

Разделители тысяч и проценты

- Разделители разрядов:
- f"{1234567:,}" → "1,234,567"
- f"{1234567:_}" → "1_234_567"
- Проценты (умножает на 100 и добавляет %):
- f"{0.758:.1%}" → "75.8%"
- f"{0.034:.2%}" → "3.40%"

консоль Python

```
>>> total = 1_234_567
>>> f"Добыча: {total:,} т"
'Добыча: 1,234,567 т'

>>> wcut = 0.258
>>> f"Обводнённость: {wcut:.1%}"
'Обводнённость: 25.8%'
```

Выравнивание и ширина поля

- {значение:ФЛАГШИРИНА} задаёт ширину колонки:
- :<N — влево :>N — вправо :^N — по центру
- Заполнитель: f"{42:0>4}" → "0042".
- f"{42:*^6}" → "**42**".
- Таблицы: числа вправо, текст влево.

консоль Python

```
>>> for name, r in [("СКВ-1",12.5),("СКВ-
2",8.3)]:
...     print(f"{name:<8} {r:>6.1f}")
...
СКВ-1      12.5
СКВ-2       8.3

>>> f"Скважина №{15:03d}"
'Скважина №015'
```

Приведение типов: int(), float(), str(), bool()

- int(x) «42» → 42, int(3.9) = 3 (отсечение!)
- float(x) «3.14» → 3.14, float(5) = 5.0
- str(x) 42 → «42», 3.14 → «3.14»
- bool(x) 0 → False, 42 → True
- Основное: результат input() → число.

консоль Python

```
# str → float → int
>>> s = "42.9"
>>> f = float(s)   # 42.9
>>> i = int(f)     # 42

>>> int(3.7)
3       # не 4!
>>> round(3.7)
4
```

int() отсекает, round() округляет

- int(x) — отбрасывает дробную часть:
- int(3.1) = 3, int(3.9) = 3, int(-3.9) = -3.
- round(x) — математическое округление:
- round(3.1) = 3, round(3.9) = 4.
- Для аналитики важно не перепутать при подсчётах.

консоль Python

```
>>> int(3.9)
3          # отсекает
>>> round(3.9)
4          # округляет
>>> int(-3.9)
-3         # к нулю
>>> int("42")
42
```

Типичные ошибки приведения

- int("abc") → ValueError int("12.5") → ValueError
- float("abc") → ValueError
- Строку с точкой — только в float: float("12.5") = 12.5.
- int("12.5") = ошибка ✗
- Пробелы мешают — помогает .strip().

```
консоль Python
ОК
>>> float("3.14")
3.14
ОШИБКИ
>>> int("abc")
ValueError: invalid literal
>>> int("12.5")
ValueError
>>> float(" 12.5 ".strip())
12.5
```

Валидация ввода: зачем

- Валидация — проверка данных перед использованием.
- Пользователь введёт текст вместо числа, отрицательное значение,
- данные выйдут за реалистичный диапазон.
- Плохая программа «падает», хорошая — сообщает и даёт исправить.

консоль Python

```
# без валидации
>>> rate = float(input("Дебит: "))
Дебит: abc
ValueError  # ← упала

# с валидацией (try/except позже)
>>> while True:
...     s = input("Дебит: ")
...     s = s.replace(",", ".")
...     try:
...         rate = float(s); break
...     except ValueError:
...         print("Введите число!")
```

Проверка типа и диапазона

- Шаги: получить строку → не пустая → цифры → int → диапазон.
- .isdigit() — состоит ли строка только из цифр.
- Для дробного: .replace('.', '', 1).isdigit().
- try/except ValueError разберём в лекции 13.

код

```
answer = input("Глубина, м: ")
if answer.strip() == "":
    print("Ошибка: пустой ввод")
elif not answer.isdigit():
    print("Введите целое число")
else:
    depth = int(answer)
    if depth <= 0:
        print("Должна быть > 0")
    elif depth > 12000:
        print("Нереалистично")
    else:
        print(f"Глубина: {depth} м — ОК")
```

КЕЙС

Консольный калькулятор скважины

- Соберём всё: название, дебит нефти, дебит воды.
- Расчёт: общий объём и обводнённость.
- Вывод: отформатированная карточка скважины.
- Используем: input(), float(), f-строки, :.1f, :.1%.

код

```
name = input("Название: ")
oil  = float(input("Нефть, т/сут: "))
water= float(input("Вода, т/сут: "))

total = oil + water
wcut  = water / total if total>0 else 0

print(f"Скважина:      {name}")
print(f"Нефть:          {oil:.1f}")
print(f"Всего жидкости: {total:.1f}")
print(f"Обводнённость:  {wcut:.1%}")
```

ЗАКРЕПИМ

Частые ошибки и как их избежать

TypeError «42» + 1 → int(«42») + 1.

input() возвращает str — для чисел оборачивайте в int()/float().

int vs round int(3.9) = 3, round(3.9) = 4.

float 0.1 + 0.2 ≠ 0.3 — сравнивайте с допуском.

f-строка забыли префикс f — переменная не подставится.

валидация неверный ввод должен обрабатываться, а не «ронять» программу.

Итоги лекции

1. Арифметика: + − * / // % **, сокращённые += и модуль math.
2. print() — sep и end; input() — всегда возвращает str.
3. f-строки — стандарт форматирования: f"{значение:формат}".
4. Формат чисел: :.Nf точность, :, разделители, :% проценты, :>N ширина.
5. Приведение: int(), float(), str(), bool(); int() отсекает, round() округляет.
6. Валидация защищает программу от неверного ввода.

Вопрос 1 из 6

Что вернёт выражение 17 / 5? А 17 // 5? Чем они отличаются?

Вопрос 2 из 6

Почему input("Число: ") + input("Ещё: ") может выдать «1020», а не 30?

Вопрос 3 из 6

Как вывести число π с тремя знаками после запятой с помощью f-строки?

Вопрос 4 из 6

Что делает оператор %? Приведите пример из нефтегазовой практики.

Вопрос 5 из 6

Будет ли ошибкой int("12.5")? Как получить из "12.5" целое число 12?

Вопрос 6 из 6

Чем int(3.9) отличается от round(3.9)?